

PepLLM: ESM-Guided Llama for Structured Protein–Peptide Binding Interface Analysis

Hao Qian¹, Shikui Tu^{1*}, Lei Xu¹

¹ Centre for Cognitive Machines and Computational Health (CMaCH),
School of Computer Science, Shanghai Jiao Tong University, Shanghai,
China.

*Corresponding author(s). E-mail(s): tushikui@sjtu.edu.cn;

Abstract

Protein–peptide interactions are central to cellular regulation and peptide-based drug discovery, yet existing computational methods mainly focus on interaction classification, binding-site prediction, or peptide binder generation. These formulations provide limited insight into the physicochemical mechanisms that determine how a peptide binds to a protein. In this work, we introduce **PepLLM**, an instruction-tuned framework for structured protein–peptide interface understanding. Given protein–peptide sequences, PepLLM generates a machine-readable JSON annotation describing multiple interface properties, including peptide burial state, hydrogen-bond density, salt-bridge presence, hotspot residues, hydrophobicity, and electrostatic complementarity. To support this task, we construct a new protein–peptide interface dataset by integrating structural interface analysis, solvent-accessible surface area computation, hydrophobic burial estimation, electrostatic potential calculation, and redundancy-aware data splitting. PepLLM connects a pretrained ESM encoder with a LLaMA decoder through a nonlinear modality adapter. The adapted ESM residue embeddings are injected into the LLaMA prompt as continuous soft tokens via placeholder-token replacement, enabling the decoder to generate structured interface annotations under instruction tuning. By moving beyond single-label prediction toward multi-property and mechanism-aware generation, PepLLM establishes a new task and modeling paradigm for interpretable protein–peptide interface analysis.

1 Introduction

Protein-peptide interactions are fundamental to a wide range of biological processes, including signal transduction, immune recognition, transcriptional regulation, and enzyme inhibition. Short peptides often bind to protein surfaces through transient and context-dependent interfaces, making them attractive candidates for therapeutic design and molecular intervention. Compared with protein-protein complexes, however, protein-peptide interactions are more difficult to characterize computationally: peptides are highly flexible, their binding poses can vary substantially across targets, and their binding affinity is determined by a mixture of geometric, chemical, and electrostatic factors. A useful computational model should therefore not only predict whether a peptide binds to a protein, but also explain how the interface is formed.

Recent years have witnessed rapid progress in computational modeling of protein-peptide interactions. Existing methods have addressed peptide-binding site prediction, peptide-protein interaction classification, residue-level interface prediction, and peptide binder generation [1–4]. In parallel, structure-based geometric learning models have improved the prediction of protein functional sites and molecular interfaces from 3D structures [5, 6]. More recently, protein language models and protein-oriented large language models have shown strong potential for protein representation learning, protein function prediction, and protein sequence generation [7–10]. Despite these advances, most existing methods are still designed around a narrow prediction format, such as binary interaction labels, binding residue scores, or generated binder sequences. They do not directly produce a structured and interpretable description of the physicochemical mechanisms underlying a protein-peptide interface.

This limitation is important because protein-peptide recognition is inherently multi-faceted. A peptide may be deeply buried in a pocket or only weakly attached to a protein surface. Its stability may be dominated by hydrogen bonds, salt bridges, hydrophobic burial, electrostatic complementarity, or a small number of hotspot residues. These properties are routinely considered by structural biologists when interpreting docking poses or experimental complex structures, but they are rarely unified into a single machine-learning task. Consequently, there remains a gap between task-specific peptide-protein predictors and the kind of mechanism-aware interface understanding required for downstream applications such as peptide drug screening, docking result interpretation, and interface engineering.

In this work, we introduce a new task, *structured protein-peptide interface understanding*. Given a protein-peptide complex represented through sequence-level embeddings, the model is required to generate a structured JSON object describing multiple interface properties, including peptide burial state, hydrogen-bond density, salt-bridge presence, hotspot residues, hydrophobicity, and electrostatic complementarity. This formulation differs from conventional classification or residue-scoring tasks in two aspects. First, the output is multi-property and mechanism-oriented rather than a single label. Second, the prediction is expressed as a constrained, machine-readable object, making it directly usable for automated evaluation, downstream filtering, and scientific reasoning.

To support this task, we construct a new protein-peptide interface dataset from complex structures. For each complex, we derive interface annotations by combining

structural interface analysis, solvent-accessible surface area computation, hydrophobic burial estimation, electrostatic potential calculation, and hotspot residue extraction. These heterogeneous structural signals are then converted into unified JSON annotations. To reduce sequence-level leakage, we perform redundancy-aware data splitting by clustering protein sequences and assigning train, validation, and test sets at the cluster level. The resulting dataset provides structured supervision for learning interpretable protein-peptide interface properties rather than only predicting interaction existence.

We further propose **PepLLM**, an instruction-tuned protein-language model for structured protein-peptide interface understanding. PepLLM connects a pretrained ESM encoder with a pretrained LLaMA decoder through a lightweight nonlinear modality adapter. The ESM encoder produces residue-level protein-peptide representations, which are projected into the LLaMA embedding space and injected into the LLaMA prompt by replacing reserved placeholder-token embeddings. The LLaMA decoder is then trained under teacher forcing to generate the target JSON annotation. In this way, PepLLM treats protein-peptide sequence representations as continuous soft tokens inside an instruction-following language model, combining the biological knowledge of protein language models with the structured generation ability of large language models.

Our main contributions are summarized as follows:

- We formulate *structured protein-peptide interface understanding* as a new task that requires unified prediction of multiple physicochemical interface properties in a machine-readable format.
- We construct a new protein-peptide interface dataset with annotations derived from structural interface parsing, hydrophobic burial computation, electrostatic complementarity estimation, hotspot extraction, and redundancy-aware data splitting.
- We propose **PepLLM**, an ESM-LLaMA instruction-tuning framework that injects protein sequence embeddings into a large language model through placeholder-based embedding replacement and generates structured interface annotations.
- We provide an ESM-only multi-head classification baseline to separate the effect of protein sequence representation learning from the benefit of instruction-based structured generation.

Together, our work moves protein-peptide modeling beyond binary interaction prediction and binding-site localization toward interpretable, multi-property, and mechanism-aware biomolecular reasoning.

2 Method

We propose **PepLLM**, an ESM-LLaMA instruction-tuning framework for structured protein-peptide interface understanding. PepLLM bridges pretrained protein sequence representations and large language model generation through a lightweight modality adapter and a placeholder-based embedding injection mechanism. The model first encodes the concatenated protein-peptide sequence with ESM, projects the residue-level hidden states into the LLaMA embedding space, and replaces reserved

placeholder-token embeddings in the LLaMA prompt with the projected protein embeddings. The LLaMA decoder is then trained to generate a valid JSON object containing multiple physicochemical properties of the protein-peptide interface.

2.1 Dataset Preparation

Each protein-peptide complex was represented by a structure file named in the form `<pdbid>_<pepChain>_<protChain>.pdb`, from which the peptide chain and protein chain were parsed directly. For each complex, the peptide and protein sequences were extracted from the first structural model using Bio.PDB peptide builders, and the final sequence input was constructed as

$$x = x_{\text{protein}}, :, x_{\text{peptide}}.$$

Samples with missing protein or peptide sequences were discarded. We further removed complexes with protein length greater than 500 residues, peptide length smaller than 2 residues, or a serialized target longer than 2000 tokens under the `cl100k_base` tokenizer. Only samples with valid electrostatic complementarity statistics were retained.

Interface annotation from structural analysis.

For each complex, PISA was first run in `--as-is` mode to identify protein-peptide interfaces and to export detailed interface reports. The PISA outputs were parsed to obtain buried solvent-accessible area, total solvent-accessible area, hydrogen bonds, salt bridges, interfacing residues, and key stabilizing or destabilizing residues. For an interface with buried areas B_1 and B_2 on the two partners, the interface area was estimated as

$$A_{\text{int}} = \begin{cases} \frac{B_1 + B_2}{2}, & B_1 > 0 \text{ and } B_2 > 0, \\ \max(B_1, B_2), & \text{otherwise.} \end{cases}$$

The peptide-like partner was assigned as the shorter partner, using residue count when available and atom count as a fallback. The peptide burial ratio was defined as

$$r_{\text{burial}} = \frac{B_{\text{peptide}}}{A_{\text{peptide}}},$$

where B_{peptide} is the buried area of the peptide and A_{peptide} is its total solvent-accessible area. The peptide binding pose was discretized as

$$\text{burial_state} = \begin{cases} \text{deep}, & r_{\text{burial}} \geq 0.60, \\ \text{partial}, & 0.30 \leq r_{\text{burial}} < 0.60, \\ \text{surface}, & r_{\text{burial}} < 0.30. \end{cases}$$

Hydrogen bonds and salt bridges were normalized by the interface area:

$$d_{\text{HB}} = 100 \cdot \frac{n_{\text{HB}}}{A_{\text{int}}}, \quad d_{\text{SB}} = 100 \cdot \frac{n_{\text{SB}}}{A_{\text{int}}}.$$

The hydrogen-bond density label was set to **high** if $d_{\text{HB}} \geq 1.0$, **low** if $0 < d_{\text{HB}} < 1.0$, and **none** otherwise. Salt bridges were labeled as **present** if $d_{\text{SB}} > 0$ and **absent** otherwise. Hotspot information was collected from the PISA key-residue blocks by counting stabilizing and destabilizing residues on both the protein and peptide sides.

Hydrophobic interface annotation.

Hydrophobic burial was computed with FreeSASA. For each complex, the protein chain, peptide chain, and full complex were evaluated separately to obtain total, polar, and apolar solvent-accessible surface areas. The buried apolar area was computed as

$$\Delta A_{\text{nonpolar}} = (A_{\text{prot,apolar}}^{\text{mono}} + A_{\text{pep,apolar}}^{\text{mono}}) - (A_{\text{prot,apolar}}^{\text{complex}} + A_{\text{pep,apolar}}^{\text{complex}}).$$

The corresponding hydrophobic free-energy contribution was estimated as

$$\Delta G_{\text{hydro}} = -\gamma \Delta A_{\text{nonpolar}}, \quad \gamma = 0.007 \text{ kcal mol}^{-1} \text{ \AA}^{-2}.$$

To obtain a categorical hydrophobicity label, we also computed the fraction of buried area that was apolar:

$$r_{\text{hydro}} = \frac{\Delta A_{\text{nonpolar}}}{(A_{\text{prot,total}}^{\text{mono}} + A_{\text{pep,total}}^{\text{mono}}) - (A_{\text{prot,total}}^{\text{complex}} + A_{\text{pep,total}}^{\text{complex}})}.$$

The hydrophobicity label was assigned as

$$\text{hydrophobicity} = \begin{cases} \text{hydrophobic}, & r_{\text{hydro}} \geq 0.60, \\ \text{mixed}, & 0.40 \leq r_{\text{hydro}} < 0.60, \\ \text{polar}, & r_{\text{hydro}} < 0.40. \end{cases}$$

Electrostatic complementarity annotation.

Electrostatic complementarity was computed from chain-specific Poisson–Boltzmann electrostatic potentials. Each complex was split into its two chains, and each chain was converted from PDB to PQR using `pdb2pqr` with the AMBER force field at pH 7.0. APBS input boxes were generated automatically from the PQR coordinates, and APBS was used to compute electrostatic potential maps for each chain. Molecular surface vertices were generated by MSMS. Interface surface points were defined as surface vertices within 4.0, \AA of any atom from the opposite chain.

The electrostatic potential of chain *B* was sampled on the interface surface of chain *A*, and vice versa. Nearest-neighbor surface-point pairs within 3.5, \AA were retained. Given paired potential values $\phi_{B \rightarrow A}$ and $\phi_{A \rightarrow B}$, the Pearson correlation was computed as

$$\rho = \text{corr}(\phi_{B \rightarrow A}, \phi_{A \rightarrow B}),$$

and electrostatic complementarity was defined as

$$\text{EC} = -\rho.$$

Thus, negative correlation between opposing electrostatic potentials corresponds to positive complementarity. We also computed the fraction of opposite-sign potential pairs,

$$f_{\text{opp}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left\{ \phi_{B \rightarrow A}^{(i)} \phi_{A \rightarrow B}^{(i)} < 0 \right\}.$$

Cases with fewer than five valid interface points or fewer than five valid paired points were treated as invalid. The final electrostatic label was assigned from EC and f_{opp} as follows:

$$C_{\text{EC}} = \begin{cases} \text{none}, & f_{\text{opp}} < 0.55, \quad \text{EC} < 0.33, \\ \text{weak}, & f_{\text{opp}} < 0.55, \quad 0.33 \leq \text{EC} < 0.50, \\ \text{moderate}, & f_{\text{opp}} < 0.55, \quad \text{EC} \geq 0.50, \\ \text{weak}, & 0.55 \leq f_{\text{opp}} \leq 0.64, \quad \text{EC} < 0.33, \\ \text{moderate}, & 0.55 \leq f_{\text{opp}} \leq 0.64, \quad 0.33 \leq \text{EC} < 0.50, \\ \text{strong}, & 0.55 \leq f_{\text{opp}} \leq 0.64, \quad \text{EC} \geq 0.50, \\ \text{moderate}, & f_{\text{opp}} > 0.64, \quad \text{EC} < 0.33, \\ \text{strong}, & f_{\text{opp}} > 0.64, \quad \text{EC} \geq 0.33. \end{cases}$$

Target construction.

For each retained complex, the structural annotations were merged into a single JSON object. In the default setting, the model was trained to generate only the coarse interface summary:

```
{
  "burial_state": "surface|partial|deep",
  "hbonds": {
    "n_hbonds": 0,
    "density": "none|low|high"
  },
  "salt_bridges": {
    "n_salt_bridges": 0,
    "presence": "absent|present"
  },
  "hotspots": {
    "n_hotspots": 0
  },
  "hydrophobicity": "polar|mixed|hydrophobic",
  "electrostatic_complementarity": "none|weak|moderate|strong"
}
```

An optional full-task setting additionally included hydrogen-bond pairs, salt-bridge pairs, and stabilizing or destabilizing hotspot residues on the protein and peptide sides. Each final JSONL record contained the PDB identifier, protein sequence, peptide

sequence, and the above target annotations. When multiple PISA interfaces were parsed for a complex, the first parsed interface was used for the downstream JSONL record.

Redundancy-aware data split.

To reduce sequence-level redundancy across splits, filtered protein sequences were written to FASTA using the PDB identifier as the sequence name. The proteins were clustered with MMseqs2 using `--min-seq-id 0.3`, `-c 0.8`, `--cov-mode 1`, `-s 7.5`, and `--cluster-mode 2`. Splits were then assigned at the cluster level rather than the individual-example level. Clusters with at least 1000 members were assigned to the training set. The remaining clusters were bucketed by cluster size and randomly split with seed 42 into training, validation, and test clusters using ratios 0.60, 0.23, and 0.17, respectively. Buckets with fewer than 10 clusters were assigned to training to avoid unstable small-bucket splits. The resulting split files were saved both as cluster-level and flattened example-level JSON files.

Batch construction for ESM-LLaMA training.

The dataset class loaded the JSONL records into memory and built an identifier map from `pdbid` to example index. During collation, the protein and peptide sequences were concatenated with a colon separator and tokenized by the ESM tokenizer. If the concatenated sequence exceeded `max_sequence_length`, a random contiguous segment of length `max_sequence_length` was sampled. ESM inputs were truncated to `max_sequence_length` residues and right-padded to `max_sequence_length + 2` tokens, accounting for the beginning-of-sequence and end-of-sequence tokens.

For the language-model input, the target annotation was serialized as compact JSON with sorted keys. The user prompt contained the text `Sequence embeddings:` followed by a repeated reserved placeholder token. The number of placeholder tokens was matched to the number of non-padding ESM sequence tokens, so that the model could inject the ESM hidden states at the corresponding positions in the LLaMA token sequence. The prompt was formatted with the LLaMA chat template and left-padded. The target JSON was appended with the LLaMA end-of-sequence token, tokenized without adding an extra BOS token, and right-padded.

During training, the model input consisted of the concatenated prompt and target tokens. The loss was applied only to the target JSON tokens: all prompt-token labels and all padding-token labels were set to -100 . Therefore, the model was optimized to generate the structured interface annotation conditioned on the protein-peptide sequence embeddings and the instruction prompt. During inference, only the formatted prompt and ESM sequence tokens were provided, and the serialized target was retained only for evaluation.

2.2 Model Architecture and Training

After constructing the protein-peptide interface annotation dataset, we trained an instruction-following multimodal language model that maps protein-peptide sequence representations to structured interface descriptions. The overall framework consists of three modules: a pretrained ESM protein encoder, a nonlinear modality adapter, and a

pretrained LLaMA causal language model decoder. The ESM encoder extracts residue-level contextual representations from the protein-peptide sequence, the adapter projects these representations into the LLaMA embedding space, and the LLaMA decoder generates the target interface annotation in JSON format.

ESM encoder.

For each input complex, the protein sequence and peptide sequence were concatenated and tokenized by the ESM tokenizer. Given an input sequence

$$s = (s_1, s_2, \dots, s_L),$$

the ESM encoder produces contextual hidden states

$$H^{\text{ESM}} = \text{ESM}(s) \in \mathbb{R}^{B \times L \times d_{\text{ESM}}},$$

where B is the batch size, L is the padded sequence length, and d_{ESM} is the hidden size of the ESM model. The ESM model was used without an additional pooling layer, so residue-level hidden states were preserved for downstream injection into the language model.

Modality adapter.

Because the hidden dimension of ESM differs from the input embedding dimension of LLaMA, we introduced a two-layer nonlinear adapter to align the two representation spaces. For each residue hidden state h_i^{ESM} , the adapter computes

$$u_i = \text{Dropout} \left(\text{GELU} \left(W_1 h_i^{\text{ESM}} + b_1 \right) \right),$$

$$z_i = \text{Dropout} \left(\text{GELU} \left(W_2 u_i + b_2 \right) \right),$$

followed by ℓ_2 normalization:

$$\tilde{z}_i = \frac{z_i}{\|z_i\|_2}.$$

The adapted representation is therefore

$$Z = (\tilde{z}_1, \tilde{z} * 2, \dots, \tilde{z} * L) \in \mathbb{R}^{B \times L \times d_{\text{LLaMA}}},$$

where d_{LLaMA} is the hidden size of the LLaMA decoder. In our implementation, the adapter intermediate dimension was set to 2048.

Embedding-level injection into LLaMA.

The adapted ESM representations were injected into LLaMA through a placeholder-token replacement mechanism. Specifically, the user prompt contained a reserved placeholder token repeated once for each non-padding ESM sequence token. Let

$$Y = (y_1, y_2, \dots, y_T)$$

denote the tokenized LLaMA input, including the system instruction, user prompt, placeholder tokens, and, during training, the target JSON sequence. The standard LLaMA embedding layer first maps these tokens to

$$E = \text{Embed}_{\text{LLaMA}}(Y) \in \mathbb{R}^{B \times T \times d_{\text{LLaMA}}}.$$

For positions corresponding to the reserved placeholder token, the original token embeddings were replaced by the adapted ESM embeddings:

$$\hat{E}_{b,t} = \begin{cases} Z_{b,k(t)}, & y_{b,t} = \langle \text{placeholder} \rangle, \\ E_{b,t}, & \text{otherwise.} \end{cases}$$

where $k(t)$ indexes the corresponding valid ESM sequence position. The ESM attention mask was used to ensure that only non-padding ESM hidden states were inserted into the LLaMA context. This design allows the LLaMA decoder to attend to protein-peptide sequence representations as continuous soft tokens, while the surrounding instruction tokens remain standard textual embeddings.

Autoregressive decoder.

The modified embedding sequence $\hat{E}$ was passed to the LLaMA causal decoder:

$$p_{\theta}(Y_{\text{out}} \mid s, Y_{\text{prompt}}) = \text{LLaMA}(\hat{E}),$$

where Y_{prompt} contains the system and user instructions, and Y_{out} is the serialized JSON annotation. The output JSON contains interface-level properties including peptide burial state, hydrogen-bond count and density, salt-bridge count and presence, hotspot count, hydrophobicity, and electrostatic complementarity. In the full-task setting, the output additionally contains residue-pair and hotspot-residue details.

Parameter-efficient instruction tuning.

The main ESM-LLaMA model was trained using parameter-efficient fine-tuning. Pre-trained ESM and LLaMA weights were loaded from their respective checkpoints, and LoRA modules were inserted into the LLaMA decoder. LoRA was applied to the self-attention projections

$$q_{\text{proj}}; k_{\text{proj}}; v_{\text{proj}}; o_{\text{proj}}$$

and the feed-forward projections

$$\text{gate}_{\text{proj}}; \text{up}_{\text{proj}}; \text{down}_{\text{proj}}.$$

The LoRA rank was set to $r = 32$, the LoRA scaling factor was set to $\alpha = 2r$, and the LoRA dropout rate was set to 0.1. Unless otherwise stated, the modality adapter was also trainable and saved together with the LoRA parameters. This strategy updates only a small number of task-specific parameters while retaining the pretrained sequence and language knowledge of ESM and LLaMA.

ESM backbone	facebook/esm2_t36_3B_UR50D
LLaMA backbone	meta-llama/Meta-Llama-3.1-8B-Instruct
Numerical precision	bfloat16
Batch size per GPU	2
Gradient accumulation steps	8
Learning rate	2×10^{-4}
Optimizer	Adam
Number of epochs	100
Learning-rate scheduler	StepLR
Scheduler decay factor	0.8
Random seed	42
LoRA rank	32

Training objective.

Training was performed with teacher forcing. For each example, the decoder input contained the prompt followed by the target JSON sequence. The labels for prompt tokens and padding tokens were set to -100 , so the loss was applied only to the JSON target tokens. The model was optimized using the standard causal language modeling objective:

$$\mathcal{L}_{\text{LM}} = -\frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \log p_{\theta}(y_t \mid y_{<t}, s).$$

where $\mathcal{T}$ denotes the set of target-token positions. This objective encourages the model to generate valid structured interface annotations conditioned on the instruction prompt and the ESM-derived protein-peptide embeddings.

Distributed training protocol.

Training was implemented with PyTorch DistributedDataParallel using the NCCL backend on a single multi-GPU node. The dataset was partitioned across GPUs with a distributed sampler; training batches were shuffled at each epoch, whereas validation batches were not shuffled. Each dataloader used four CPU workers, pinned memory, and dropped incomplete batches during training and validation.

Unless otherwise specified, the main ESM-LLaMA model was trained with the following hyperparameters: The loss was divided by the number of gradient-accumulation steps before backpropagation, so each optimizer update corresponded to multiple micro-batches. Gradient norm clipping was supported and disabled by default. After each epoch, the learning-rate scheduler was stepped and the model was evaluated on the validation split under teacher forcing. Training loss, validation loss, learning rate, and gradient norm were recorded with TensorBoard. Adapter checkpoints and optimizer/scheduler states were saved periodically.

Inference and generation.

During inference, only the prompt and protein-peptide sequence were provided. The ESM encoder first produced residue-level hidden states, the modality adapter projected them into the LLaMA embedding space, and the projected embeddings replaced the placeholder-token embeddings in the prompt. The resulting prompt embeddings

were passed to the LLaMA generation function to autoregressively produce the output JSON. Before inference, the PEFT adapter was merged into the base model for generation.

Generation was performed on the test split with a distributed sampler and without shuffling. Unless otherwise specified, generation used greedy decoding with one beam, no sampling, a maximum of 2000 newly generated tokens, temperature 1.0, top- p equal to 1.0, and top- k equal to 50. The end-of-sequence token ID was set to 128009 and the padding token ID was set to 128002. Predictions and ground-truth JSON strings were decoded with the LLaMA tokenizer and saved as per-rank JSON files for downstream metric computation.

ESM-only classification baseline.

To assess the contribution of the LLaMA decoder and instruction-based generation, we also implemented an ESM-only classification baseline. This baseline uses the same ESM encoder but removes the LLaMA decoder. The ESM residue-level hidden states are first mean-pooled with the sequence attention mask:

$$h_{\text{pool}} = \frac{\sum_{i=1}^L m_i h_i^{\text{ESM}}}{\sum_{i=1}^L m_i},$$

where m_i is the ESM attention mask. The pooled representation is then passed to a multi-head adapter. Each prediction head is a two-layer nonlinear adapter and outputs logits for one categorical interface property. The output dimensions of the five heads were set to

$$[3, 3, 2, 3, 4],$$

corresponding to the class numbers of burial state, hydrogen-bond density, salt-bridge presence, hydrophobicity, and electrostatic complementarity, respectively.

For the ESM-only baseline, the training loss was the average cross-entropy over all prediction heads:

$$\mathcal{L}_{\text{cls}} = \frac{1}{K} \sum_{k=1}^K \text{CE} \left(\hat{y}^{(k)}, y^{(k)} \right),$$

where $K = 5$. The baseline was trained using the same distributed training framework. Unless otherwise specified, it used bfloat16 precision, Adam optimization, learning rate 2×10^{-4} , 100 epochs, StepLR decay factor 0.8, gradient accumulation of 8 steps, and random seed 42. The batch size per GPU was set to 4 for the ESM-only baseline. During inference, the predicted class for each head was obtained by taking the argmax over the corresponding logits.

3 Experiments

3.1 Experimental Setup

We evaluate PepLLM on structured protein-peptide binding interface analysis. Each input consists of a protein sequence and a peptide sequence, and each output is a structured JSON object containing multiple interface properties.

Table 1 Dataset statistics used in the current experiments.

Split	Number of examples
Train	~32K
Validation	~1.8K
Test	~1.8K

In the current experiments, we focus on a five-field classification setting covering the following categorical attributes:

- `burial_state`;
- `electrostatic_complementarity`;
- `hbonds_density`;
- `hydrophobicity`;
- `salt_bridges_presence`.

These fields correspond to peptide burial state, electrostatic complementarity, hydrogen-bond density, hydrophobic character, and salt-bridge presence, respectively.

The dataset is constructed from protein-peptide complex structures and split using protein-sequence clustering to reduce sequence-level leakage. After filtering long sequences and clustering proteins with MMseqs2, the current dataset contains approximately 32K training examples, 1.8K validation examples, and 1.8K test examples. The protein sequence length is restricted to be less than 500 residues, the peptide sequence length is restricted to be less than 50 residues, and the serialized JSON target is restricted to be shorter than 2000 tokens.

Models

We compare two model families. The first is **PepLLM**, a generative model that encodes the protein-peptide sequence pair with a pretrained protein sequence encoder, projects the sequence embeddings through an MLP adapter, and feeds the adapted embeddings into a general-purpose LLM decoder. The decoder is trained to generate the target interface annotation as a JSON string.

The second model is an **ESM+adapter** baseline. This baseline removes the LLM decoder and directly predicts the five categorical labels using a multi-head adapter on top of the ESM sequence representation. This comparison is designed to separate the contribution of structured LLM generation from the protein sequence representation itself.

Metrics

For the generative PepLLM model, we report the JSON pass rate and field-wise accuracy. The JSON pass rate measures whether the generated string can be parsed as a

Table 2 PepLLM results on the five-field classification setting. Accuracy is reported in percentage. “Avg.” denotes the mean over the five categorical fields.

Metric / Field	Epoch 1	Epoch 10
JSON pass rate	100.00	99.73
<code>burial.state</code>	60.93	64.40
<code>electrostatic.complementarity</code>	34.08	35.18
<code>hbonds.density</code>	42.57	48.11
<code>hydrophobicity</code>	50.05	52.26
<code>salt.bridges.presence</code>	58.07	61.36
Avg. categorical accuracy	49.14	52.26

valid output object:

$$\text{PassRate} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{ValidJSON}(\hat{y}_i)],$$

For each categorical field f , we compute field accuracy:

$$\text{Acc}_f = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\hat{y}_{i,f} = y_{i,f}].$$

For count-valued fields, including the number of hydrogen bonds, hotspots, and salt bridges, we additionally report mean absolute error (MAE):

$$\text{MAE}_f = \frac{1}{N} \sum_{i=1}^N |\hat{y}_{i,f} - y_{i,f}|.$$

3.2 Main Results

Table 2 reports the performance of PepLLM after one epoch and ten epochs of training. The model produces syntactically valid JSON outputs with a very high pass rate throughout training. At Epoch 1, the pass rate is 100.00%, indicating that the model quickly learns the required output schema. At Epoch 10, the pass rate remains high at 99.73%.

Across the five categorical fields, PepLLM improves from an average accuracy of 49.14% at Epoch 1 to 52.26% at Epoch 10. The largest gains are observed for `hbonds.density`, which improves from 42.57% to 48.11%, and `burial.state`, which improves from 60.93% to 64.40%. In contrast, `electrostatic.complementarity` remains the most challenging attribute, suggesting that electrostatic interface labels may require more accurate supervision or richer structural context.

Table 3 reports the MAE of count-valued JSON fields. From Epoch 1 to Epoch 10, the MAE decreases for hydrogen-bond count and hotspot count. The hotspot

Table 3 PepLLM results on count-valued JSON fields. Lower MAE is better.

Count field	Epoch 1	Epoch 10
<code>hbonds.n_hbonds</code>	3.09	2.92
<code>hotspots.n_hotspots</code>	4.15	3.69
<code>salt_bridges.n_salt_bridges</code>	1.56	1.61
Avg. MAE	2.93	2.74

Table 4 Comparison between PepLLM and the ESM+adapter classification baseline on the five categorical fields. Accuracy is reported in percentage.

Field	PepLLM	ESM+adapter
<code>burial_state</code>	64.40	62.63
<code>electrostatic_complementarity</code>	35.18	38.32
<code>hbonds.density</code>	48.11	48.09
<code>hydrophobicity</code>	52.26	62.10
<code>salt_bridges.presence</code>	61.36	59.93
Avg. categorical accuracy	52.26	54.21

count MAE decreases from 4.15 to 3.69, suggesting that the model benefits from additional training when predicting coarse interaction intensity. The salt-bridge count MAE remains similar, increasing slightly from 1.56 to 1.61, which is consistent with the sparsity and imbalance of salt-bridge events.

3.3 Comparison with ESM-only Classification

We further compare PepLLM with the ESM+adapter baseline on the same five categorical fields. Table 4 summarizes the results. The ESM+adapter baseline achieves an average accuracy of 54.21%, slightly higher than PepLLM’s 52.26%. However, the two models exhibit different strengths. PepLLM performs better on `burial_state`, `hbonds.density`, and `salt_bridges.presence`, whereas the ESM+adapter baseline performs better on `hydrophobicity` and `electrostatic_complementarity`.

This comparison suggests that simple discriminative heads can be competitive for single-field classification, especially for global physicochemical attributes. In contrast, PepLLM is designed for structured generation and can produce a unified JSON output containing both categorical labels and count-valued interaction information. Therefore, although the discriminative baseline is strong for isolated classification fields, PepLLM provides a more general output format for interpretable interface analysis.

3.4 Discussion

The preliminary experiments lead to three observations. First, PepLLM can reliably generate valid JSON outputs, showing that a general-purpose LLM can be

adapted to structured biomolecular interface analysis when conditioned on protein-peptide sequence embeddings. Second, PepLLM improves over training on most categorical fields and reduces count-field MAE, demonstrating that the model learns nontrivial correlations between sequence-pair embeddings and interface annotations. Third, the ESM+adapter baseline remains competitive on some isolated categorical labels, especially hydrophobicity and electrostatic complementarity, suggesting that future versions of PepLLM should combine structured generation with stronger discriminative or auxiliary objectives.

Overall, these results validate the feasibility of the proposed task and framework. While the current experiment is still preliminary, it demonstrates that protein-peptide binding interface analysis can be formulated as a structured generation problem rather than only as independent classification tasks.

4 Conclusion

We presented **PepLLM**, a new dataset-driven instruction-tuning framework for structured protein-peptide interface understanding. Unlike prior work that mainly focuses on interaction classification, binding-site localization, residue-level contact prediction, or peptide binder generation, PepLLM is designed to generate a structured description of the underlying interface mechanisms. By combining structural interface parsing, hydrophobic burial estimation, electrostatic complementarity computation, and cluster-based data splitting, we constructed a dataset that provides physicochemically grounded supervision for protein-peptide interface analysis.

PepLLM connects pretrained protein sequence representations with large language model generation. Through a lightweight modality adapter and placeholder-based embedding injection, ESM-derived residue embeddings are inserted into the LLaMA context as continuous soft tokens. The model is trained to generate valid JSON annotations that summarize multiple interface properties in a unified format. This framework demonstrates a practical way to repurpose large language models for structured biomolecular prediction while retaining the biological knowledge of protein language models.

More broadly, PepLLM suggests a new direction for computational protein science: using language models not only to classify biological sequences or generate candidate binders, but also to produce structured, interpretable, and mechanism-aware descriptions of molecular interactions. Such a formulation can support downstream applications including peptide drug screening, docking result interpretation, interface engineering, and automated scientific hypothesis generation. Future work may extend PepLLM by incorporating explicit 3D structural encoders, jointly modeling multiple candidate binding poses, and validating the generated interface annotations against experimental binding and mutagenesis data.

References

- [1] Abdin, O., Nim, S., Wen, H., Kim, P.M.: PepNN: a deep attention model for the identification of peptide binding sites. *Communications Biology* **5**(1), 503 (2022) <https://doi.org/10.1038/s42003-022-03445-2>

- [2] Chandra, A., Sharma, A., Dehzangi, I., Tsunoda, T., Sattar, A.: PepCNN deep learning tool for predicting peptide binding residues in proteins using sequence, structural, and language model features. *Scientific Reports* **13**(1), 20882 (2023) <https://doi.org/10.1038/s41598-023-47624-5>
- [3] Jin, X., Chen, Z., Yu, D., Jiang, Q., Chen, Z., Yan, B., Qin, J., Liu, Y., Wang, J.: TPepPro: a deep learning model for predicting peptide–protein interactions. *Bioinformatics* **41**(1), 708 (2025) <https://doi.org/10.1093/bioinformatics/btae708>
- [4] Chen, L.T., Quinn, Z., Dumas, M., Peng, C., Hong, L., Lopez-Gonzalez, M., Mestre, A., Watson, R., Vincoff, S., Zhao, L., Wu, J., Stavrand, A., Schaepeers-Cheu, M., Wang, T.Z., Srijay, D., Monticello, C., Vure, P., Pulugurta, R., Pertsemlidis, S., Kholina, K., Goel, S., DeLisa, M.P., Chi, J.-T.A., Truant, R., Aguilar, H.C., Chatterjee, P.: Target sequence-conditioned design of peptide binders using masked language modeling. *Nature Biotechnology* **44**, 1002–1010 (2026) <https://doi.org/10.1038/s41587-025-02761-2> . Published online 13 August 2025
- [5] Tubiana, J., Schneidman-Duhovny, D., Wolfson, H.J.: ScanNet: an interpretable geometric deep learning model for structure-based protein binding site prediction. *Nature Methods* **19**(6), 730–739 (2022) <https://doi.org/10.1038/s41592-022-01490-7>
- [6] Krapp, L.F., Abriata, L.A., Cortés Rodriguez, F., Dal Peraro, M.: PeSTo: parameter-free geometric deep learning for accurate prediction of protein binding interfaces. *Nature Communications* **14**(1), 2175 (2023) <https://doi.org/10.1038/s41467-023-37701-8>
- [7] Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., Smetanin, N., Verkuil, R., Kabeli, O., Shmueli, Y., Santos Costa, A., Fazel-Zarandi, M., Sercu, T., Candido, S., Rives, A.: Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science* **379**(6637), 1123–1130 (2023) <https://doi.org/10.1126/science.ade2574>
- [8] Zhuo, L., Chi, Z., Xu, M., Huang, H., Zhao, J., Zheng, H., He, C., Mao, X.-L., Zhang, W.: ProtLLM: An interleaved protein-language LLM with protein-as-word pre-training. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8950–8963. Association for Computational Linguistics, Bangkok, Thailand (2024). <https://doi.org/10.18653/v1/2024.acl-long.484> . <https://aclanthology.org/2024.acl-long.484/>
- [9] Lv, L., Lin, Z., Li, H., Liu, Y., Cui, J., Chen, C.Y.-C., Yuan, L., Tian, Y.: ProLLaMA: A protein large language model for multitask protein language processing. *IEEE Transactions on Artificial Intelligence* **7**(2), 642–653 (2026) <https://doi.org/10.1109/TAI.2025.3564914>

- [10] Liu, N., Sun, C., Ji, T., Tian, J., Tang, J., Wu, Y., Lan, M.: EvoLlama: Enhancing LLMs’ understanding of proteins via multimodal structure and sequence representations. arXiv preprint arXiv:2412.11618 (2024) [arXiv:2412.11618](https://arxiv.org/abs/2412.11618) [cs.LG]

Appendix A Cross-Docking Dataset Construction for Peptide-Protein Complexes

A.1 Overview

Given a clustered peptide-protein complex dataset, the goal is to construct cross-docked peptide-protein poses for downstream LLM training. Each complex is identified as

$$x = \text{pdbid_peptideChain_receptorChain}.$$

For each cluster C_g , each member can serve as a donor template and each other member can serve as an acceptor target. A directed cross-docking pair is defined as

$$(d, a), \quad d, a \in C_g, \quad d \neq a,$$

where d provides the template peptide and a provides the target receptor.

A.2 Step 1: Standardization and Quality Control

For each complex x , the structure file is parsed from the input PDB/mmCIF directory. The peptide chain and receptor chain are extracted according to the identifier

$$x = \text{pdbid_peptideChain_receptorChain}.$$

Only the first model is used if the structure contains multiple models. For each complex, three standardized PDB files are written:

$$\text{complex_raw.pdb}, \quad \text{peptide.pdb}, \quad \text{receptor.pdb}.$$

The following quality-control features are computed for both peptide and receptor chains:

$$\begin{aligned} \ell &= \text{number of amino-acid residues,} \\ n_{\text{nonstd}} &= \text{number of non-standard residues,} \\ n_{\text{missBB}} &= \#\{r : \{N, CA, C\} \not\subseteq \text{Atoms}(r)\}, \\ n_{\text{break}} &= \#\{(r_i, r_{i+1}) : \|CA_i - CA_{i+1}\|_2 > 4.5 \text{ \AA}\}. \end{aligned}$$

Each complex is assigned a status:

$$\text{status}(x) \in \{\text{ok}, \text{repairable}, \text{failed}\}.$$

A complex is marked as failed if the structure is missing, cannot be parsed, has missing required chains, or has an empty peptide or receptor chain. It is marked as repairable if backbone atoms are missing but the chains can still be exported.

The outputs of this step are:

$$\text{complex_table.csv}, \quad \text{pair_table.csv}, \quad \text{cluster_table.csv}, \quad \text{qc_report.csv}.$$

For each cluster C_g , the initial directed pair set is

$$\mathcal{P}_g = \{(d, a) : d, a \in C_g, d \neq a\},$$

with size

$$|\mathcal{P}_g| = |C_g|(|C_g| - 1).$$

A.3 Step 1.5: Pair Pre-screening and Top- K Re-ranking

Because the full directed pair table can be close to $O(n^2)$, a lightweight two-stage ranking procedure is used before docking.

For each standardized complex x , peptide and receptor sequences are extracted. The receptor binding site is defined by heavy-atom contacts to the native peptide:

$$S_x = \left\{ i \in R_x : \min_{j \in P_x} \min_{\alpha, \beta} \|r_{i, \alpha} - p_{j, \beta}\|_2 \leq 6.0 \text{ \AA} \right\}.$$

The contact site is then expanded by a sequence padding window:

$$S_x^+ = \{i - k, \dots, i + k : i \in S_x, k = 1\}.$$

A donor-quality score is computed from the Step 1 QC features:

$$\begin{aligned} q_d = \text{clip}_{[0,1]} & \left(1 - \Delta_{\text{status}} - 0.03 \min(5, n_{\text{pepMissBB}}) - 0.02 \min(10, n_{\text{recMissBB}}) \right. \\ & - 0.01 \min(10, n_{\text{pepNonstd}}) - 0.005 \min(10, n_{\text{recNonstd}}) - 0.05 \min(5, n_{\text{pepBreak}}) \\ & \left. - 0.03 \min(5, n_{\text{recBreak}}) \right). \end{aligned}$$

where

$$\Delta_{\text{status}} = \begin{cases} 0, & \text{ok,} \\ 0.15, & \text{repairable,} \\ 1.0, & \text{failed.} \end{cases}$$

For each acceptor a , all possible donors d in the same cluster are cheaply pre-scored. Let v_x^P be the amino-acid composition vector of the peptide sequence, and v_x^S be the composition vector of the receptor binding-site sequence. Define

$$\rho_{d,a}^P = \frac{\min(\ell_d^P, \ell_a^P)}{\max(\ell_d^P, \ell_a^P)}, \quad \rho_{d,a}^S = \frac{\min(\ell_d^S, \ell_a^S)}{\max(\ell_d^S, \ell_a^S)}.$$

A candidate pair is valid only if

$$\begin{aligned} d & \neq a, \\ \rho_{d,a}^P & \geq 0.60, \\ |\ell_d^P - \ell_a^P| & \leq 15, \end{aligned}$$

$$\rho_{d,a}^S \geq 0.50,$$

and both peptide and site lengths are non-zero.

The cheap pre-score is

$$s_{d,a}^{\text{pre}} = 0.35 \cos(v_d^S, v_a^S) + 0.25 \cos(v_d^P, v_a^P) + 0.15 \rho_{d,a}^P + 0.15 \rho_{d,a}^S + 0.10 q_d.$$

For each acceptor, only the top M donors by s^{pre} are kept, where the default is

$$M = 100.$$

These candidates are then re-ranked using exact global sequence identities:

$$I_{d,a}^S = \text{SeqId}(S_d, S_a), \quad I_{d,a}^P = \text{SeqId}(P_d, P_a).$$

The final ranking score is

$$s_{d,a}^{\text{final}} = 0.45 s_{d,a}^{\text{pre}} + 0.30 I_{d,a}^S + 0.20 I_{d,a}^P + 0.05 q_d.$$

For each acceptor, the final top K donors are selected:

$$\mathcal{T}_a = \text{TopK}_d(s_{d,a}^{\text{final}}), \quad K = 25.$$

The outputs are:

`complex_feature_table.csv`, `pair_table_ranked.csv`, `pair_table_topk.csv`.

A.4 Step 2: Local Binding-Site Alignment and Initial Model Construction

For each selected donor-acceptor pair (d, a) , the standardized receptor and peptide structures are loaded.

The donor and acceptor native binding sites are recomputed using the same heavy-atom contact rule:

$$S_d = \left\{ i \in R_d : \min_{j \in P_d} \min_{\alpha, \beta} \|r_{i,\alpha}^d - p_{j,\beta}^d\|_2 \leq 6.0 \text{ \AA} \right\},$$

$$S_a = \left\{ i \in R_a : \min_{j \in P_a} \min_{\alpha, \beta} \|r_{i,\alpha}^a - p_{j,\beta}^a\|_2 \leq 6.0 \text{ \AA} \right\}.$$

The donor and acceptor receptor sequences are globally aligned:

$$A(R_d, R_a) \rightarrow \pi_{d \rightarrow a},$$

where $\pi_{d \rightarrow a}$ maps donor receptor residue indices to acceptor receptor residue indices.

Mapped site pairs are constructed as

$$\mathcal{M}_{d,a} = \{(i, \pi_{d \rightarrow a}(i)) : i \in S_d, \pi_{d \rightarrow a}(i) \in S_a, CA_i, CA_{\pi(i)} \text{ exist}\}.$$

If

$$|\mathcal{M}_{d,a}| < 4,$$

the pipeline attempts a fallback using all full-chain mapped C_α pairs. If the fallback also contains too few pairs, the cross-docking pair is rejected.

A rigid-body superposition is computed by minimizing

$$(R^*, t^*) = \arg \min_{R, t} \sum_{(i,j) \in \mathcal{M}_{d,a}} \|RCA_i^d + t - CA_j^a\|_2^2.$$

The donor peptide is transferred into the acceptor receptor coordinate system:

$$P_{d \rightarrow a} = R^* P_d + t^*.$$

The initial cross-docked model is then defined as

$$M_{d,a}^0 = R_a \cup P_{d \rightarrow a}.$$

Importantly, the acceptor native peptide is saved only as a reference and is not used for model construction:

$$P_a^{\text{native}} \notin M_{d,a}^0.$$

For each pair, the following metadata are recorded:

local_RMSD, receptor_seq_identity, peptide_seq_identity, $n_{\text{clashResidues}}$, $n_{\text{clashAtomPairs}}$.

The main output is:

`pair_summary.csv`,

and, when model writing is enabled,

`initial_model.pdb`.

A.5 Step 3: ADCP Local-Docking Job Selection

Only successful Step 2 pairs with existing initial models are considered for ADCP local docking. A pair is accepted if it satisfies:

$$\ell_d^P \leq 50,$$

$$|\mathcal{M}_{d,a}| \geq 4,$$

$$\text{local_RMSD}_{d,a} \leq 8.0 \text{ \AA},$$

$$n_{\text{clashResidues}} \leq 12.$$

If an extrapolation ratio is available, the additional filter is

$$\text{extrap_ratio}_{d,a} \leq 0.50.$$

Optionally, the pipeline can require

$$\text{alignment_mode} = \text{local_site},$$

and can reject trivial native-like cases where

$$\text{SeqId}(R_d, R_a) = 1 \quad \text{and} \quad \text{SeqId}(P_d, P_a) = 1.$$

Accepted jobs are written to

$$\text{jobs.csv}.$$

A.6 Step 4: Preparing PDB Inputs for HPC Docking

For each accepted job, the initial model is copied into a flat docking-input directory:

$$\text{step4}/\{\text{pair_id}\}.\text{pdb}.$$

For large-scale HPC transfer, the PDB files can be split into file lists, packed into tar chunks, transferred to the remote cluster, and extracted into the remote Step 4 directory:

$$\text{step4_tar_chunks}/\text{chunk}.*.\text{tar} \rightarrow \text{remote}/\text{step4}/.$$

A.7 Step 5: ADCP Local Docking

For each input complex $M_{d,a}^0$, the ADCP driver performs the following operations.

First, the complex is split into receptor and peptide PDB files:

$$M_{d,a}^0 \rightarrow \text{receptor.pdb}, \quad \text{peptide.pdb}.$$

The peptide sequence is extracted from the peptide PDB:

$$P_{d \rightarrow a} \rightarrow \text{seq}(P_d).$$

Both receptor and peptide are protonated:

$$\text{receptor.pdb} \xrightarrow{\text{reduce}} \text{receptorH.pdb},$$

$$\text{peptide.pdb} \xrightarrow{\text{reduce}} \text{peptideH.pdb}.$$

They are then converted to PDBQT:

$$\text{receptorH.pdb} \xrightarrow{\text{prepare_receptor}} \text{receptorH.pdbqt},$$

`peptideH.pdb` $\xrightarrow{\text{prepare_ligand}}$ `peptideH.pdbqt`.

A local docking target is built around the current peptide pose:

`agfr (receptorH.pdbqt, peptideH.pdbqt, padding = 4.0 Å) → job.trg`.

ADCP is then run as

`adcp (-t job.trg, -s peptide_sequence, -N 20, -n 100000, -ref peptideH.pdb)`.

The output is a ranked set of docked peptide poses:

`job_cdock_ranked_*.pdb`.

On SLURM, the job table is distributed across workers by row index:

$$\text{worker}(i) = i \bmod N_{\text{tasks}}.$$

The provided batch script launches

$$5 \text{ nodes} \times 10 \text{ tasks per node} = 50 \text{ workers},$$

with 6 CPU cores per worker.

A.8 Step 6: Post-processing and Final Complex Reconstruction

After ADCP, each docked peptide pose is merged back with the corresponding protein receptor.

The protein chain is extracted from the Step 4 initial complex, and the docked peptide chain is extracted from the ADCP output. The final chain identifiers are normalized as

$$\text{peptide chain} = A, \quad \text{protein chain} = B.$$

The final reconstructed complex is

$$M_{d,a,k}^{\text{dock}} = P_{d,a,k}^{\text{dock}} \cup R_a,$$

where k indexes the ADCP ranked pose.

The final outputs are stored in

`step6/{pair_id}/{ranked_pose}.pdb`.

A.9 LLM Training Record Construction

Each final training example can be represented as a structured record:

$$z = (\text{input}_{d,a}, \text{target}_{d,a,k}, \text{metadata}_{d,a,k}).$$

The input can include:

$$\text{input}_{d,a} = \{\text{seq}(R_a), \text{seq}(P_d), M_{d,a}^0, \text{cluster_id}, \text{donor_id}, \text{acceptor_id}, s_{d,a}^{\text{final}}\}.$$

The target can be the final docked complex or the docked peptide coordinates:

$$\text{target}_{d,a,k} = M_{d,a,k}^{\text{dock}} \quad \text{or} \quad P_{d,a,k}^{\text{dock}}.$$

The metadata should include:

$$\{\text{local_RMSD}, \text{site_pair_n}, \text{peptide_length}, \text{clash_counts}, \text{ADCP_rank}, \text{source_cluster}\}.$$

To avoid information leakage, train/validation/test splitting should be performed at the cluster level:

$$C_g \cap C_h = \emptyset, \quad g \in \mathcal{G}_{\text{train}}, \quad h \in \mathcal{G}_{\text{test}}.$$

All cross-docking pairs derived from the same cluster should remain in the same dataset split.

Overall, the pipeline generated approximately 30,000 directed peptide–protein cross-docking pairs, providing a large-scale training corpus for LLM-based peptide–protein binding and docking modeling.

A.10 Implementation Notes

1. Step 1.5 writes both `pair_table_ranked.csv` and `pair_table_topk.csv`. For large-scale docking, the top- K file should normally be used as the Step 2 input.
2. Step 2 must be executed with model writing enabled; otherwise `initial_model.pdb` will not exist and Step 3 will reject the job.
3. The ADCP job preparation step uses local RMSD, site-pair count, peptide length, and clash count as heuristic filters. These thresholds should be tuned for the desired balance between throughput and quality.
4. The final post-processing step assumes that the docked peptide chain in the ADCP output is chain A , and rewrites the final complex as peptide chain A and protein chain B .